Operating System Fundamentals • Chapter 01

Introduction

What an operating system is, and how systems grew from room-sized mainframes to the phone in your pocket.

Chapter 1 of the notebook

BIG QUESTION

What does an operating system actually do for you, all day long?

The whole course

Four parts, one story

PART 1

Overview

- Introduction
- Computer System Structure
- Operating System Structure

PART 2

Process Management

- Processes
- CPU Scheduling
- Deadlocks

PART 3

Storage Management

- Memory Management
- Virtual Memory *(self-study)*
- File Management *(self-study)*

PART 4

Cloud

- Introduction to Cloud Computing

On your own

Self-study topics and references



Virtual Memory

- Background
- Demand paging
- Page replacement
- Allocation of frames



File-System Interface

- File concept
- Access methods
- Directory structure
- Protection



File-System Implementation

- Structure & allocation
- Free-space management
- Directory implementation
- Recovery

References: Silberschatz, *Operating System Concepts* · Furht & Escalante, *Handbook of Cloud Computing*

Today's route

What we'll cover today

1. **What is an OS?**
   Its job and where it sits
2. **System components**
   Hardware, OS, apps, users
3. **How systems evolved**
   Batch → multiprogramming → time-sharing → desktop
4. **More than one CPU**
   Multiprocessor, distributed, clustered
5. **Special-purpose systems**
   Real-time and handheld
6. **Computing environments**
   Traditional, web-based, embedded

Part 01

What is an operating system?

THINK FIRST

Your laptop runs a browser, music and dozens of background apps on a handful of CPU cores. Who decides who gets what?

discuss for one minute before the next slide

DEFINITION

The operating system is the intermediary between you and the hardware.

You ask for things: open a file, play a video. The OS turns those requests into work the hardware can do.

Why we need one

Four jobs of every OS



Execute user programs

Load a program into memory and run it.



Solve user problems

Give programs what they need to do the user's work.



Use hardware efficiently

Keep the CPU, memory and devices busy doing useful work.



Make life easier

Hide the messy details of the hardware from users and programmers.

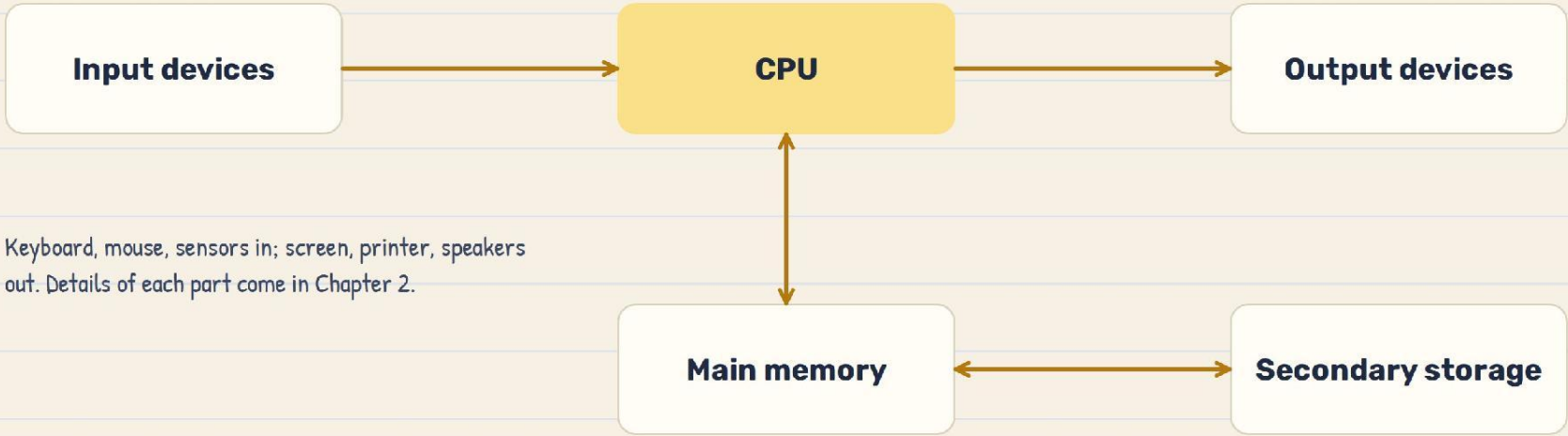
Computer system components

Four layers, the OS in the middle

Users	People, machines, other computers
Application programs	Compilers, web browsers, spreadsheets, word processors
Operating system	Controls and coordinates hardware use among programs and users
Computer hardware	CPU, memory, I/O devices, storage

Layer 1

The hardware the OS manages



Keyboard, mouse, sensors in; screen, printer, speakers out. Details of each part come in Chapter 2.

Layer 2

What the OS actually does



Resource allocator

Manages and allocates CPU time, memory, files and devices among competing programs.



Control program

Controls the execution of user programs and the operation of I/O devices, preventing errors and misuse.



The kernel

The one program running at all times. Everything else is either a system program or an application.

Layers 3 and 4

Who sits on top



Application programs

- Compilers
- Web browsers
- Spreadsheets
- Word processors
- ...anything that solves a user's problem



Users

- People
- Machines (e.g. a sensor feeding a factory controller)
- Other computers (e.g. a server calling another server)

Pop quiz · click to reveal

Which one is the kernel?

A. Your web browser

B. The one program running at all times

C. The command you type in the terminal

D. The power button

Pop quiz · click to reveal

Which one is the kernel?

A. Your web browser

B. The one program running at all times

C. The command you type in the terminal

D. The power button

Answer B. The kernel is the part of the OS that is always in memory and always running.

Part 02

How systems evolved

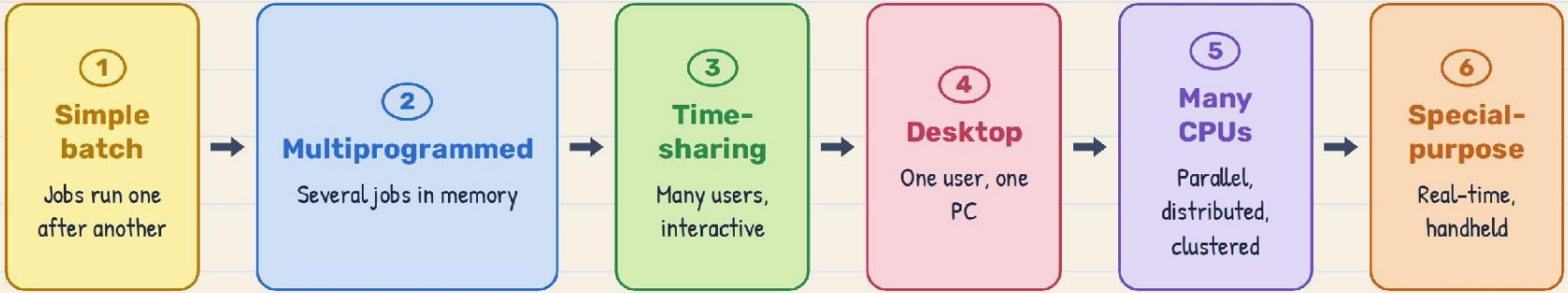
THINK FIRST

The first computers had no OS: a human operator loaded each job by hand. If you were that operator, what would you automate first?

discuss for one minute before the next slide

The family tree

Each step fixed the previous step's weakness



Every idea on this line is still inside a modern OS. Windows, Linux and Android all multiprogram, time-share and run on multiple cores.

Mainframe systems

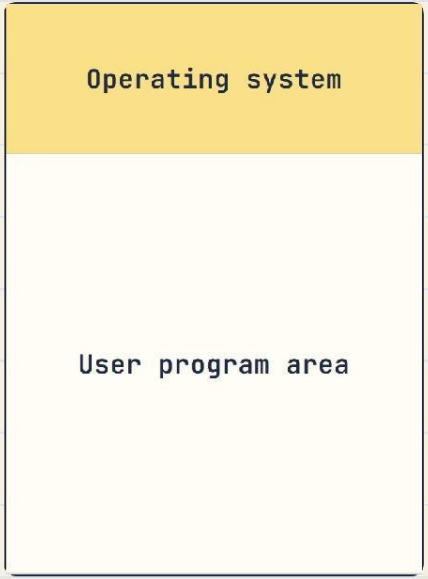
Simple batch: queue the jobs, run them in turn

Batch similar jobs

Group jobs that need the same setup to cut setup time.

Automatic job sequencing

Control passes from one job to the next automatically: the first rudimentary operating system.



Memory layout: one job at a time

Mainframe systems

Multiprogramming: never let the CPU sit idle

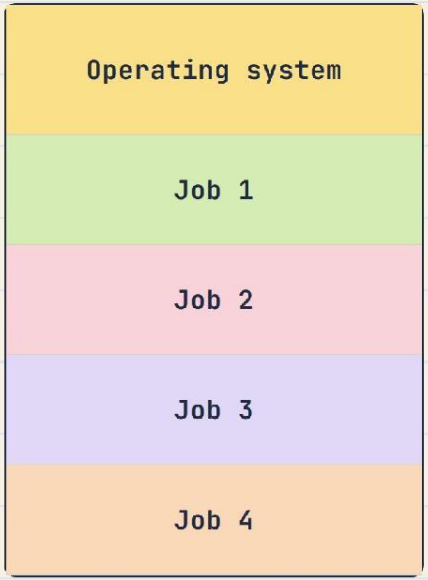
The idea

Keep several jobs in main memory at once. When one waits for I/O, the CPU switches to another.

Kitchen analogy

A chef with four dishes doesn't stare at the oven. While one bakes, they chop for the next.

↺ *we'll reuse this idea in Ch 4 & 5*



Memory layout: several jobs

Mainframe systems

Time-sharing:
many users, one computer, all feel alone

Rapid switching

The CPU switches between users' jobs so fast that every user feels served.

Only jobs in memory run

The CPU goes to a job only if it is in memory. Jobs are swapped between memory and disk (details in Ch 7).

Interactive

When a command finishes, the OS reads the next "control statement" from the user's keyboard.

Always online

Data and code must be available on-line for users to reach.

Side by side

Three mainframe styles

	Simple batch	Multiprogrammed	Time-sharing
Jobs in memory	One	Several	Several (+ swapped to disk)
User interaction	None	None	Yes, on-line
Main goal	Cut setup time	Keep CPU busy	Fast response to users
Key trick	Automatic job sequencing	Switch on I/O wait	Switch very frequently

Desktop systems

One computer, one person



Personal computers

- Dedicated to a single user
- I/O: keyboard, mouse, display, small printer
- Runs Windows, macOS, UNIX, Linux



Design priorities

- User convenience and responsiveness
- Borrows technology from larger OSes
- Single user, so less need for advanced CPU-utilisation or protection features

Pop quiz • click to reveal

In a multiprogrammed system, what does the CPU do while Job 1 waits for the disk?

A. Sits idle until the disk finishes

B. Switches to another job in memory

C. Restarts Job 1 from the beginning

D. Asks the user what to do

Pop quiz • click to reveal

In a multiprogrammed system, what does the CPU do while Job 1 waits for the disk?

A. Sits idle until the disk finishes

B. Switches to another job in memory

C. Restarts Job 1 from the beginning

D. Asks the user what to do

Answer B. The whole point of multiprogramming is that the CPU is never idle while there is other work in memory.

Part 03

More than one CPU

THINK FIRST

Two cooks in one shared kitchen, or two restaurants swapping recipes by phone: which pair is tightly coupled?

discuss for one minute before the next slide

Tightly vs loosely coupled

Multiprocessor vs distributed

ONE KITCHEN

Multiprocessor (parallel)

- More than one CPU in close communication
- **Tightly coupled:** CPUs share memory and a clock
- Communicate through shared memory
- Wins: throughput, economy, reliability

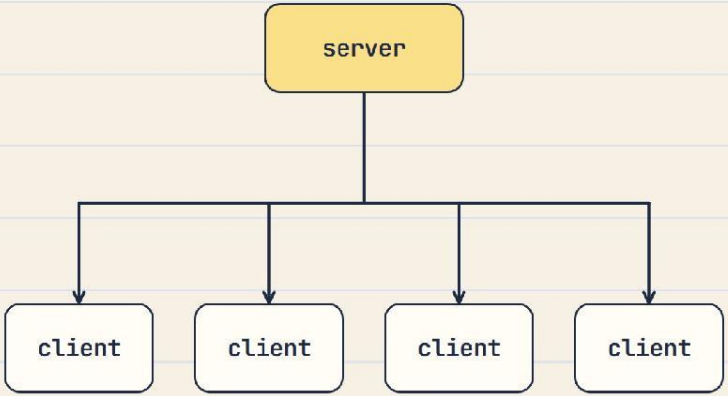
MANY RESTAURANTS

Distributed

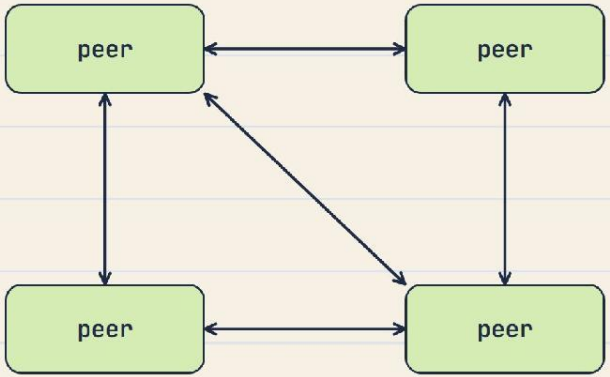
- Computation spread over several physical machines
- **Loosely coupled:** each has its own local memory
- Communicate over buses or network lines
- Wins: resource sharing, speed-up (load sharing), reliability

Distributed systems

Built on a network: LAN or WAN



Client-server: one side serves, the others ask



Peer-to-peer: every node can serve and ask

Clustered systems

Several computers, shared storage, high reliability



HOT STANDBY

Asymmetric clustering

One server runs the applications; the others stand by, ready to take over if it fails.



ALL HANDS

Symmetric clustering

All N hosts run the applications and watch each other. No hardware sits idle.

Football analogy: asymmetric = a substitute goalkeeper waiting on the bench; symmetric = every player on the pitch covering for each other.

Pop quiz • click to reveal

"Processors share memory and a clock." Which system is this?

A Distributed system

B Tightly coupled multiprocessor

C Asymmetric cluster

D Handheld system

Pop quiz • click to reveal

"Processors share memory and a clock." Which system is this?

A Distributed system

B Tightly coupled multiprocessor

C Asymmetric cluster

D Handheld system

Answer B. Sharing memory and a clock is the definition of tightly coupled (parallel) systems.

Part 04

Special-purpose systems

THINK FIRST

A car's airbag controller must react within milliseconds of a crash. Would "usually fast" be good enough?

discuss for one minute before the next slide

Real-time systems

Correct, and on time



Where you find them

- Controlling scientific experiments
- Medical imaging systems
- Industrial control systems
- Some display systems



What makes them different

- Often a control device in a dedicated application
- Well-defined, fixed time constraints
- A late answer is a wrong answer
- Comes in two kinds: hard and soft

Real-time systems

Hard vs soft

	Hard real-time	Soft real-time
Deadline	Must never be missed	A critical task gets priority until it completes
Storage	Secondary storage limited or absent; data in short-term memory or ROM	Normal storage
Time-sharing?	Conflicts with it; not supported by general-purpose OSes	Can be integrated with time-sharing
Typical use	Airbags, pacemakers, flight control	Multimedia, virtual reality; limited use in industrial robotics

Handheld systems

Small devices, big constraints



Devices

- Personal Digital Assistants (PDAs)
- Mobile phones and smartphones
- Tablets



Classic issues

- Limited memory
- Slow processors
- Small display screens

Computing environments

Where the OS runs



Traditional

PCs and servers, limited remote access.



Web-based

Client-server and web services, convenient remote access, location-less servers.



Embedded

Very limited OS features, little or no user interface, remote access.

Pop quiz · click to reveal

A car's anti-lock brake controller is best described as…

- **A** A time-sharing system
- **B** A hard real-time embedded system
- **C** A soft real-time desktop system
- **D** A peer-to-peer system

Pop quiz · click to reveal

A car's anti-lock brake controller is best described as…

- **A** A time-sharing system
- **B** A hard real-time embedded system
- **C** A soft real-time desktop system
- **D** A peer-to-peer system

Answer B. Missing a braking deadline is unacceptable (hard real-time), and it runs inside the car with no user interface (embedded).

Recap

Every system type in one table

System	Key idea	Example
Batch / multiprogrammed	Keep the CPU busy	Payroll runs on a mainframe
Time-sharing	Fast switching for many users	Terminals on a shared server
Desktop	One user, convenience first	Your laptop
Multiprocessor	Shared memory and clock	A multi-core CPU
Distributed / clustered	Separate machines cooperate	A web farm, a database cluster
Real-time / handheld	Deadlines / tight resources	Airbag, smartphone

Wrap-up

What to remember

OS JOURNAL · CHAPTER 1

- ✓ The OS is the intermediary between users and hardware.
- ✓ Multiprogramming keeps the CPU busy; time-sharing makes it interactive.
- ✓ Clusters share storage for high reliability.
- ✓ It allocates resources and controls program execution; the kernel is always running.
- ✓ Tightly coupled = shared memory and clock; loosely coupled = own memory, network.
- ✓ Real-time systems are judged on deadlines, not just correctness.

Next up *Chapter 2 · Computer System Structure: what happens inside the box*